

安装、使用产品，请阅读安装使用说明书。

请妥善保管好本手册，以便日后能随时查阅。

教学案例

四川天链机器人股份有限公司

人形机器人系统开发实验

一、培训目标

掌握基于 ROS2 系统、RS485 通信和 PID 控制的人形机器人一体化关节开发技术，能够独立完成系统搭建、程序编写与调试，实现机器人关节的精准控制。

二、培训内容

理论基础：人形机器人系统架构、一体化关节原理、PID 控制理论、ROS2 系统基础、RS485 通信原理

1. 人形机器人系统架构

关键技术：感知与决策、定位与导航、数据与模型、人机交互、结构与机构、运动与操作、通信技术等。

机器人组件：控制模组、传感器模组、执行模组、芯片模组、能源模组等。

机器人软件：开发平台、仿真平台、调参推理平台。

机器人应用：工业制造、教育科研、医疗健康、危险作业、文体娱乐等。

2. 一体化关节原理

一体化关节的核心原理在于将机器人关节所需的多个关键部件（通常是电机、减速器、驱动器和位置/力矩传感器）高度集成封装在一个紧凑、密封的单元内。共享同一个外壳、散热系统和电气连接，内部信号传输通过电路板直接完成，动力传递路径也被最大程度地缩短和优化。具有结构紧凑，重量轻，高刚性，高精度，低振动，动态响应快等优点。

3. PID 控制理论

PID 控制是工业控制中最经典、应用最广泛的控制算法之一，其核心思想是通过对系统误差（设定值与实际值之差）的比例（P）、积分（I）和微分（D）三部分进行线性组合，生成控制量来驱动被控对象，使其输出快速、稳定地跟踪设定值。

4. ROS2 系统基础

ROS2 是专为现代机器人开发设计的操作系统框架，它通过模块化工具和标准化通信协议帮助开发者快速构建复杂的机器人系统。与 ROS1 相比，ROS2 采用更先进的分布式架构，取消了中心化的主节点，转而依赖 DDS（数据分发服务）实现去中心化通信，从而提升了系统的可靠性和实时性。

在 ROS2 中，核心功能单元是节点，每个节点代表一个独立的功能模块，例如传感器驱动或运动控制。节点之间通过话题、服务和动作三种机制进行通信。话题用于异步单向数据传输，适合持续性的信息流如传感器数据；服务提供同步的请求-响应模式，适用于即时控制指令；而动作则支持长时间任务的管理，并能反馈执行状态。

5. RS-485 通信原理

RS-485 是一种异步串行通信协议，可以实现多节点通信。RS-485 通信基于差分信号，其中信息通过两根电线（通常称为 A 和 B）发送的两个互补信号传输。这两根电线之间的电压差是传达信息的因素，而不是单根电线和接地之间的电压。这使得 RS-485 系统对共模噪声具有很强的抵抗力。并且能够提高传输距离和传输速度。RS-485 协议规定，一个主节点可以与最多 32 个从节点进行通信，而各个节点之间的通信都是通过主节点进行协调的。

环境搭建：ROS2 系统安装配置、RS422 通信协议 、编程工具使用

1. ROS2 系统安装配置

首先在终端输入命令 `lsb_release -a` 查看自己的 ubuntu 版本，以 T1pro 机器人为例，对应应该下载 Iron 版本，具体的安装及配置方法见

<https://blog.csdn.net/www1994/article/details/132580833>

2. RS422 通信协议

RS-422 接口标准全称是“平衡电压数字接口电路的电气特性”，它定义了接口电路的特性。实际上还有一根信号地线，共 5 根线。由于接收器采用高输入阻抗和发送驱动器比 RS232 更强的

驱动能力，故允许在相同传输线上连接多个接收节点，最多可接 10 个节点。一个主设备 Master，其余为从设备 Slave，从设备之间不能通信，所以 RS-422 支持点对多的双向通信。接收器输入阻抗为 4k，故发端最大负载能力是 $10 \times 4k + 100 \Omega$ （终接电阻）。

3. 编程工具使用

vscode, pycharm 等

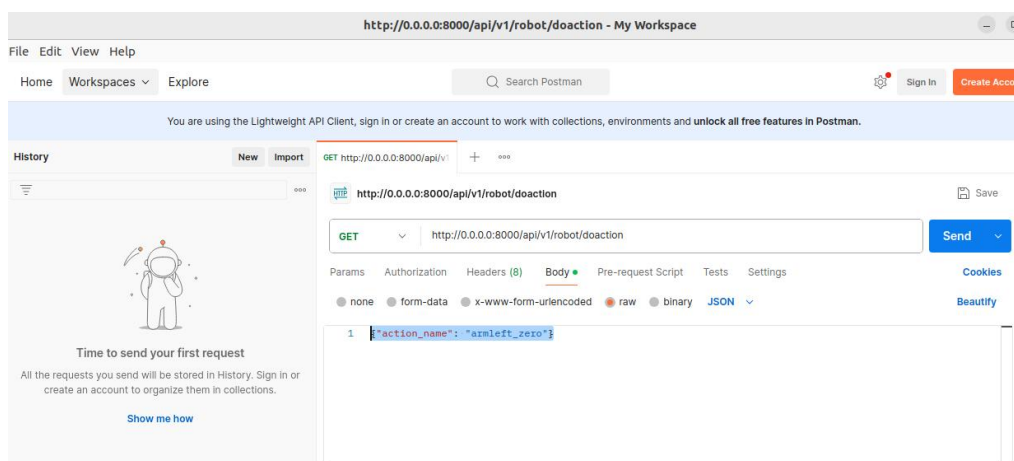
实践操作：一体化关节硬件连接与测试、422 通信编程、PID 控制算法实现、人形机器人综合项目开发

首先按亮机器人背后的电源按键，松开机器人背后的急停按钮，观察无异常情况下在终端输入 `tlpro_launch.sh` 命令，若启动成功会出现关节状态灯快速闪动。

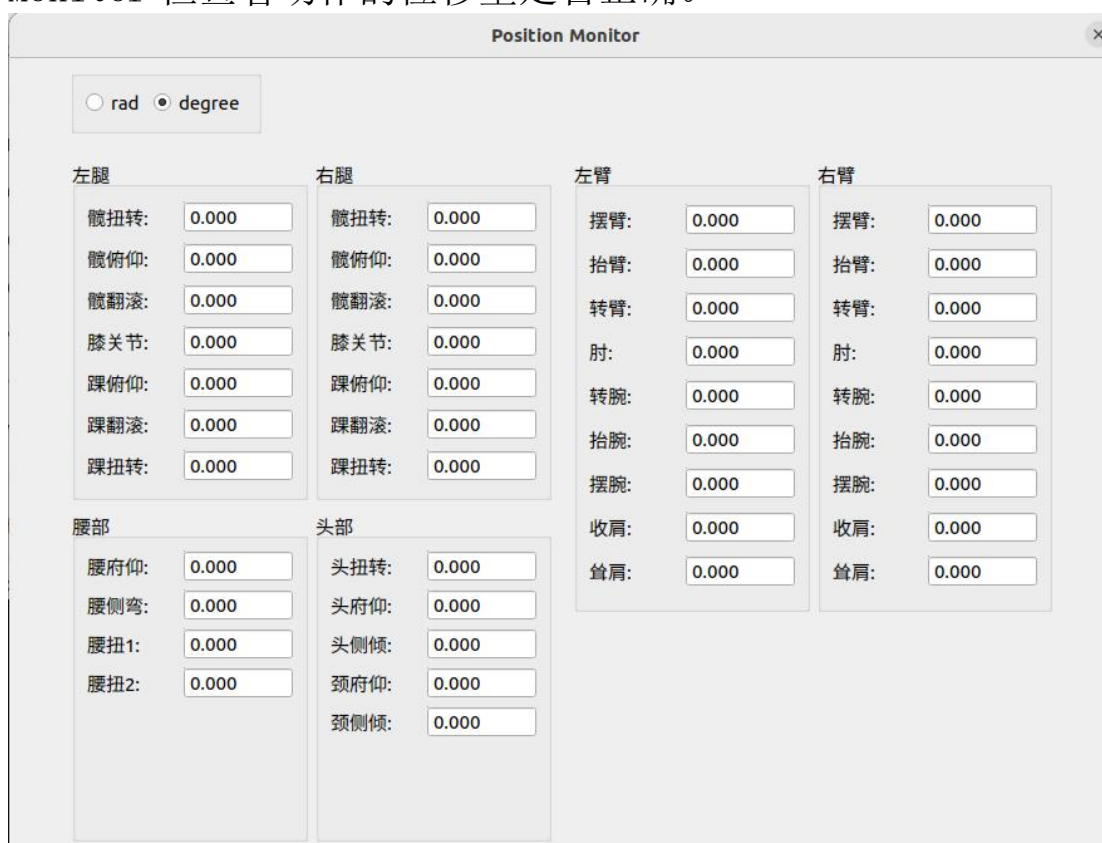
接下来开始进行机器人关节参数调试，进入 `FDR_ROBOT_API` 目录在终端用 `vscode` 打开，先修改 `third-part/robot_action_helper/src/robot_action_helper/base` 目录下文件 `common.py`，将不进行测试的关节进行屏蔽、然后在 `resource` 目录下的 `data.py` 文件进行定义需要调试关节的动作、然后将上一步的动作名字输入到 `robot_action_client` 文件里面。

然后终端输入 `tlpro_api.sh`，然后另起终端输入 `postman`，在 `post` 一栏输入 `test/client_api.py` 文件里名字为机器人动作 `http` 开头的参数

（`http://0.0.0.0:8000/api/v1/robot/doaction`），`body` 改为 `raw`，格式改为 `json`。在 `action_name` 输入刚刚定义好的动作名称，在点击 `send` 以后可查看话题 `/arm_right_joint_trajectory_controller/joint_trajectory` 可以检查控制指令是否生效。

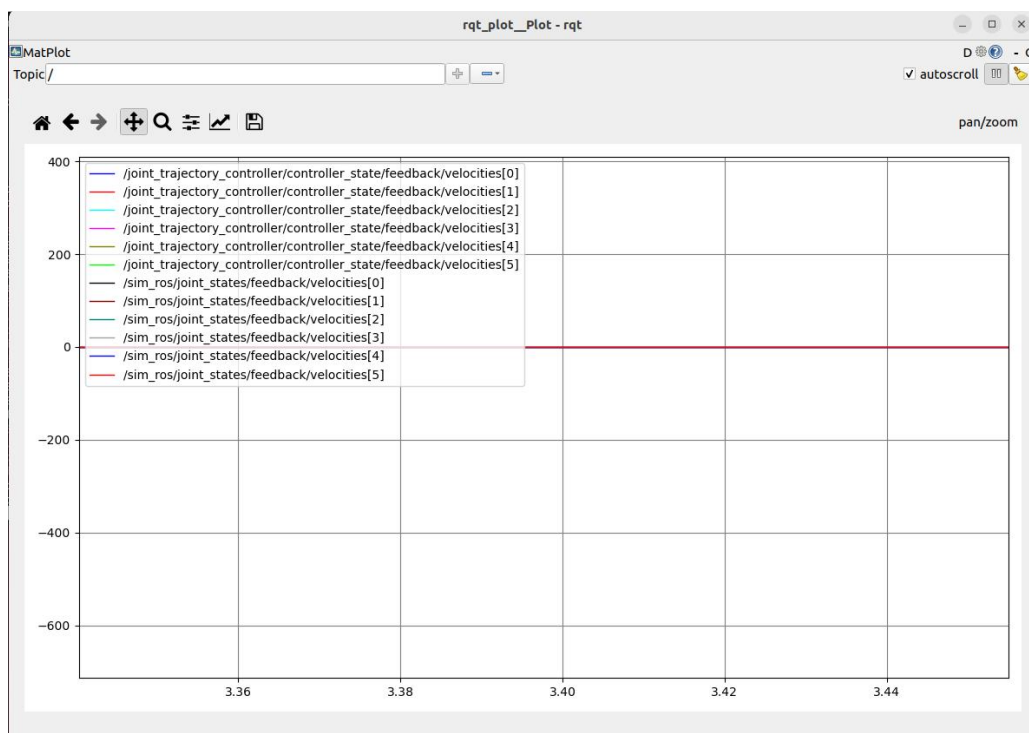


接着可以打开 robot_device_monitor.sh 脚本里 Position Monitor 栏查看动作的位移量是否正确。



在终端打开 canopen_node_monitor.sh 查看机器人的测试关节的相关控制参数，进入界面选择 scope 查看对应关节的参

数波形。



选择 kpkd 即可根据需要在修改控制参数 k_p 和 k_d 。



人形机器人通信技术应用开发实验

一、培训目标

学员通过本培训，深入理解 RS422 通信、CANopen 协议、402 协议及 485 通信原理，熟练掌握在人形机器人系统中运用这些技术进行数据传输与设备控制的方法，具备独立完成通信模块开发、调试及优化的能力，解决机器人系统通信过程中的实际问题。

二、培训内容

理论基础：422 通信原理、CANopen 协议架构、402 协议功能详解、485 通信特点与应用场景

1. RS422 通信原理

RS422 的工作原理主要涉及信号的差分传输和校验机制。在 RS422 通信中，发射端将数字信号转换成差分信号发送，接收端对接收到的差分信号进行解码并且进行校验。这种差分传输的特性保证了 RS422 在较长距离内依然有较高的信号质量和抗干扰能力。RS422 通信中，通常会使用校验位来检测数据传输的错误，并且进行必要的纠正。这种校验机制能够有效保证数据传输的准确性和完整性，为应用提供了可靠的数据通信保障。

2. CANopen 协议架构

CAN 总线本身只解决了数据“如何可靠、实时地传输”的问题（物理层 + 数据链路层），并没有规定传输的数据代表什么含义以及设备间如何协同工作。而 CANopen 是构

建在 CAN 总线之上的应用层协议和设备规范，它为基于 CAN 的系统赋予了组织规则和共同语言。

CANopen 定义了标准化的通信服务：PDO 用于高效传输实时的传感器值、控制指令等，映射自对象字典；SDO 用于读写对象字典，配置参数或传输大数据块；NMT 负责整个网络的启动、停止和状态监控；EMCY 用于紧急错误报告。通过定义设备描述文件，CANopen 确保了不同厂商设备的互操作性，让它们能在同一条 CAN 总线上按同一套规则运行。

3.402 协议功能详解

402 协议 则是在 CANopen 这个成熟平台上，为电机运动控制这个专业领域量身定制的一套详尽的行业操作规范和标准接口，402 是 CANopen 协议的子协议，它深度利用了下层提供的 CAN 的实时性、CANopen 的标准化通信与数据管理来实现复杂、高性能的运动控制功能。

主要功能：

- (1) 定义电机驱动器的状态机，如启动、停止、故障恢复等。
- (2) 支持多种操作模式，如位置控制、速度控制、扭矩控制、回零模式等。
- (3) 通过标准化的对象字典配置参数，如目标位置、最大速度、加速度等。

4.485 通信特点与应用场景

485 通讯协议是工业自动化领域常用的串行通信协议，利用差分信号传输数据，抗干扰能力强，传输距离远。其数据帧包括起始位、数据位、校验位和停止位。与 232 通讯协议相比，485 在传输距离、抗干扰、节点数目和通讯速率上有优势。常见应用包括工控系统、仪器仪表和电力系统。

技术解析：CANopen 网络拓扑结构、402 协议在伺服控制中的应用、485 通信与 CAN 通信的对比

1. CANopen 网络拓扑结构

CANopen 网络的拓扑结构包含以下几种常见形式：

（1）总线拓扑结构： 所有节点通过一条总线连接，信息通过总线广播给所有节点。这种拓扑结构适用于简单的通信场景，但随着节点数量的增加，总线上的数据冲突可能会增多，影响通信性能。

（2）星型拓扑结构： 多个节点通过集线器或交换机连接，数据通过集线器/交换机转发到目标节点，这种拓扑结构可以减少总线上的数据冲突，提高通信的可靠性。

（3）混合拓扑结构： 将总线型和星型拓扑结构相结合，适用于复杂网络环境。通过合理规划节点的连接方式，可以充分发挥 CAN 总线通信的优势。

2. 402 协议在伺服控制中的应用

（1）402 协议定义了一套标准的对象模型（如运动设备对象、DS402 对象、位置控制器对象、速度控制器对象等）和对象字典（OD）。

（2）支持多种标准化的运动控制模式：轮廓位置模式、轮廓速度模式、轮廓转矩模式等

（3）定义了标准的错误代码、报警代码以及状态字中的位来指示驱动器状态（如使能状态、警告、故障、跟随误差超限等）。

3. 485 通信与 422 通信的对比

工作模式差异：

RS-485 是半双工通信标准，任何时候只能有一点处于发送状态。它采用两线制差分传输，同一对信号线既用于发送也用于接收。

RS-422 也采用全双工工作方式，但使用四线制：两对差分信号线，一对用于发送，一对用于接收。由于收与发通道分开，RS-422 设备可以同时收发数据，适合高速双向通信系统。

网络拓扑结构：

RS-422 支持点对多通信，一个主设备可以连接多个从设备（最多 10 个），但从设备之间不能直接通信。这种拓扑结构适合主从式监控系统。

RS-485 支持多点通信，可以构建总线型拓扑结构，在一条总线上连接多个驱动器和接收器（最多 128 个）。各设备通过使能信号控制总线占用，这种拓扑结构非常适合设备联网和分布式系统。

终端匹配技术：

对于 RS-422，在 300 米以下的短距离传输时可不需终接电阻。而 RS-485 在总线两端都需要连接终端电阻，特别是在高速或长距离传输时。

开发实践： CANopen 从站与主站配置、基于 402 协议的电机控制开发、485 通信程序编写与调试

综合实验： 搭建人形机器人多通信技术融合系统，实现不同通信模块间的数据交互与协同控制

人形机器人与大模型技术应用开发实验

一、培训目标

掌握语音交互、人脸识别技术与大语言模型（LLM）在人形机器人上的应用开发方法，能够将多模态感知与大模型能力集成，实现机器人智能对话、身份识别及场景化任务处理，提升人形机器人智能化水平与交互体验。

二、培训内容

理论基础： 语音交互技术原理、人脸识别算法基础、大语言模型（LLM）架构与工作机制

1. 语音交互技术原理

首先，系统通过麦克风捕捉到人说话的声音，并清除掉背景噪音等干扰，得到比较清晰的音频信号。然后，语音识别部分（ASR）开始工作，它把这个声音信号分析成一个个微小的片段，利用强大的学习模型找出这些声音片段最可能对应的基本发音单位，再结合语言的规律，把这些发音组合成最有可能的、符合语法的文字句子。接下来，自然语言理解部分（NLU）接手，它的任务是弄明白这段文字的意思和用户想干什么。它会分析词语之间的关系，识别出关键信息，最重要的是判断用户的意图以及完成这个意图需要的具体信息。理解了用户意图后，系统需要决定怎么回应。它会根据当前的对话状态和预设的规则，决定是直接回答问题、反问确认、执行操作还是做其他事情。决定好回应内容后，自然语言生成部分负责把这个回应写成一句通顺的人话。对于简单的回应，可能直接用预设好的句子填上关键信息；复杂的回应则需要动态组织语言。最后，语音合成部分（TTS）把生成好的文字句子转换成可以播放的声音。

当机器接收到用户语音后，首先通过语音识别（ASR），是将声音转化成文字的过程，相当于耳朵。然后经过自然语言处理（NLU），是理解和处理文本的过程，相当于大脑。最后进行语音合成（TTS），是把文本转化成语音的过程，相当于嘴巴，完成一次交互。

2. 人脸识别算法基础

人脸识别算法的基础流程是：系统先在图像中找到人脸位置并框选出来，然后调整人脸角度使其大致正面朝向，接着通过深度学习模型从对齐后的人脸中提取一组独特的数字特征码，这个特征码能区分不同人且不受光线、表情等变化影响。最后将提取的特征码与数据库中预存的人脸特征码进行相似度计算，若相似度超过设定阈值，则判定为同一人，完成身份识别。整个过程依赖于大量数据训练出的模型对关键面部特征的捕捉和编码能力。

3. 大语言模型（LLM）架构与工作机制

大语言模型的核心原理是：根据前面生成的内容，预测下一个最可能出现的词，本质上是一个自回归的概率模型。实现语义理解和文本生成等自然语言处理任务的关键，在于两个方面：一是基于 Transformer 架构的强大建模能力，二是对海量语料数据的训练。而 Transformer 的核心创新在于自注意力机制——每个词都能够关注上下文中的其他词，动态融合全局信息，从而更好地理解语境，实现精准的词预测。

其中 Transformer 架构最关键的模块包括：

（1）嵌入层

将离散的文本符号（单词或子词）转换为模型可处理的连续数值向量也就是最小的语言单位（Token），并融入位置信息，为每个 Token 分配一个唯一的数字 ID。

（2）多头注意力机制（Self-Attention）

其功能是让模型在处理序列中每个 Token 时，能够动态地、并行地关注序列中所有其他相关的 Token，捕捉长距离依赖关系和复杂的上下文信息。

(3) 前馈神经网络 (Feed Forward Network - FFN)
对经过自注意力机制更新后的每个 Token 的向量表示进行独立、非线性的变换和增强，赋予模型更强的拟合复杂模式的能力。

(4) 残差连接与层归一化 (Residual Connection & Layer Normalization)
解决深层网络训练中的梯度消失/爆炸和信息丢失问题，提升训练的稳定性和效率。

(5) 输出层 (Output Layer)
将 Decoder 最终的隐藏状态转换为预测下一个 Token 的概率分布，并应用采样策略生成最终输出。

技术应用： 语音识别与合成 API 使用、人脸识别模型训练与部署、LLM 接入与调用方法

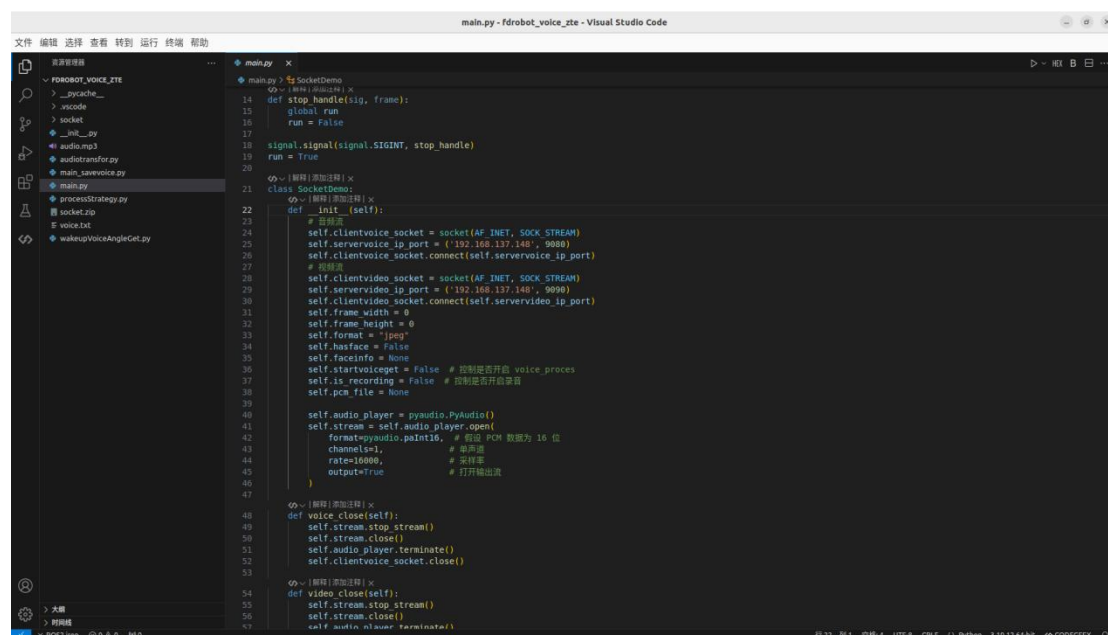
开发实践： 基于开源框架搭建语音交互模块、实现人脸识别功能与人形机器人动作联动、将集成到机器人对话系统中

系统整合： 多模态数据融合处理，开发结合语音、视觉与 LLM 的人形机器人智能交互应用

首先语音识别模块将音频流转换为文本，然后将文本传递给 LLM 对话系统，在 LLM 接收到传来的文本后会结合历史对话信息生成回复，此时通过 TTS 将 LLM 生成的回复转换为语音输出，并通过扬声器播放。

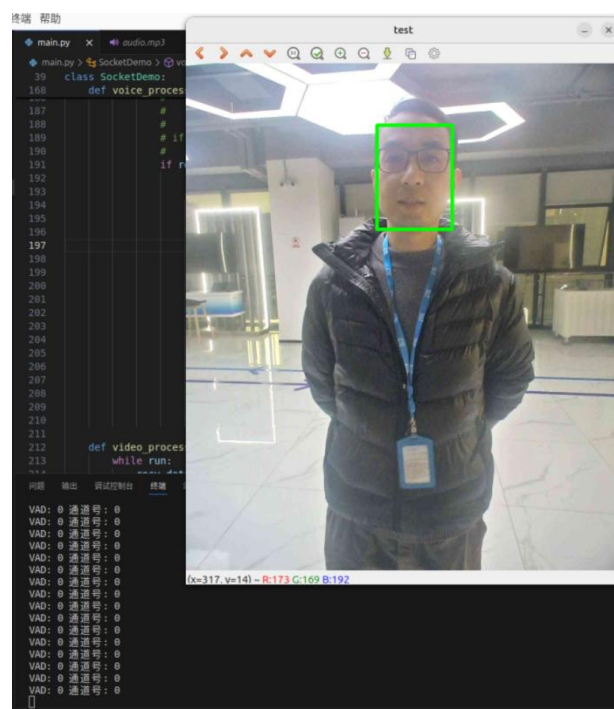
代码位于 `/home/fdrobot/fdrobot_voice_zte` 中，运行 `main.py` 代码，就可以启动获取讯飞语音的视频流和音

频流，有两个端口，端口 IP 为 192.168.137.148，9080 为音频流端口，9090 为视频流端口。



```
main.py x SocketDemo
14 def stop_handle(sig, frame):
15     global run
16     run = False
17
18 signal.signal(signal.SIGINT, stop_handle)
19 run = True
20
21 class SocketDemo:
22     def __init__(self):
23         # 音频流
24         self.clientvoice_socket = socket(AF_INET, SOCK_STREAM)
25         self.servvoice_ip_port = ('192.168.137.148', 9080)
26         self.clientvoice_socket.connect(self.servvoice_ip_port)
27         # 视频流
28         self.clientvideo_socket = socket(AF_INET, SOCK_STREAM)
29         self.servvideo_ip_port = ('192.168.137.148', 9090)
30         self.clientvideo_socket.connect(self.servvideo_ip_port)
31         self.frame_width = 0
32         self.frame_height = 0
33         self.format = "jpeg"
34         self.hasface = False
35         self.faceinfo = None
36         self.startvoiceget = False # 控制是否开启 voice process
37         self.is_recording = False # 控制是否开启录音
38         self.pcm_file = None
39
40         self.audio_player = pyaudio.PyAudio()
41         self.stream = self.audio_player.open(
42             format=pyaudio.paInt16, # 音频 PCM 数据为 16 位
43             channels=1, # 单声道
44             rate=10000, # 采样率
45             output=True # 打开输出流
46         )
47
48     def voice_close(self):
49         self.stream.stop_stream()
50         self.stream.close()
51         self.audio_player.terminate()
52         self.clientvoice_socket.close()
53
54     def video_close(self):
55         self.stream.stop_stream()
56         self.stream.close()
57         self.audio_player.terminate()
```

运行后如下图所示：



如果遇到下面问题，则需要重新运行 main.py 代码。


```
File "/usr/lib/python3/dist-packages/requests/api.py", line 115, in post
    return request("post", url, data=data, json=json, **kwargs)
File "/usr/lib/python3/dist-packages/requests/api.py", line 59, in request
    return session.request(method=method, url=url, **kwargs)
File "/usr/lib/python3/dist-packages/requests/sessions.py", line 589, in request
    resp = self.send(prepared_request, **send_kwargs)
File "/usr/lib/python3/dist-packages/requests/sessions.py", line 783, in send
    r = adapter.send(request, **kwargs)
File "/usr/lib/python3/dist-packages/requests/adapters.py", line 519, in send
    raise ConnectionError(e, request=request)
requests.exceptions.ConnectionError: HTTPConnectionPool(host='10.20.7.263', port=30125): Max retries exceeded with url: /api/request (Caused by NewConnectionError('<urllib3.connection.HTTPConnection object at 0x78df2f39ba18>: Failed to establish a new connection: [Errno 111] Connection refused'))
```

人形机器人运控技术应用开发实验

一、培训目标

学员能够熟练运用 Mujoco 和 gym 仿真平台，结合强化学习与传统控制方法，掌握机器人运动学与动力学原理，并将其应用于人形机器人运动开发，实现机器人稳定行走、复杂动作规划等功能，具备独立完成人形机器人运动仿真与实际开发的能力。

二、培训内容

理论基础： 机器人运动学正逆解、动力学方程推导；强化学习基本概念与算法原理；传统控制方法（如 PID、LQR）介绍

1. 机器人运动学正逆解

正运动学模型描述了从关节角度到末端执行器位置和姿态的映射。给定关节角度，正运动学模型可以计算出末端执行器的确切位置和方向。

逆运动学模型描述了从末端执行器位置和姿态到关节角度的映射。给定末端执行器的目标位置和姿态，逆运动学模型可以计算出实现这一目标所需的关节角度。

2. 动力学方程推导

机器人动力学的两种方法——拉格朗日方程和牛顿欧拉方程，重点讲介绍拉格朗日方程，动力学推导可参，原文链接

https://blog.csdn.net/qq_40498018/article/details/130525839

3. 强化学习基本概念与算法原理

原文链接：

https://blog.csdn.net/K_KK_KK_KK/article/details/145415957

4. 传统控制方法（如 PID、LQR）介绍

原文链接：

https://blog.csdn.net/weixin_42964413/article/details/142855497

仿真平台应用：Mujoco 仿真环境搭建与人形机器人模型导入；gym 平台基础使用与环境配置；仿真场景构建与参数设置

1. Mujoco 仿真环境搭建与人形机器人模型导入

```
(1) 在终端输入 pip install mujoco 安装 mujoco
(2) 在终端输入 python -m mujoco.viewer --
mjcf=/path/to/some/mjcf.xml      #为指定的机器人模型
文件启动可视化会话
```

2. gym 平台基础使用与环境配置；仿真场景构建与参数设置

gym 平台的环境搭建与基础使用可参照以下内容，原文链接
<https://blog.csdn.net/frigidwinter/article/details/128400818>

仿真场景构建与参数设置可参照以下内容，原文链接

https://blog.csdn.net/qq_47997583/article/details/122487670

控制算法实践：基于传统控制方法实现人形机器人基础运动控制；运用强化学习算法进行机器人运动策略训练；对比分析两种控制方法在机器人运动中的效果

综合开发实验：结合运动学动力学，利用仿真平台完成人形机器人复杂运动任务开发；从仿真到实际机器人的运动控制迁移与调试

1. 利用仿真平台完成人形机器人复杂运动任务开发（sim to sim）

（1）构建仿真环境：

在 Isaac Sim 中创建虚拟环境，例如室内场景、工厂流水线或动态障碍物区域。

导入机器人模型（URDF 或 SDF 格式），设置传感器和物理属性。

（2）训练控制算法：

使用强化学习或模仿学习算法，在仿真环境中训练机器人完成任务。

利用 GPU 并行化加速训练过程，减少计算时间。

（3）生成合成数据：

模拟不同光照条件、材质特性和传感器噪声，生成多样化的数据集，提升模型的泛化能力。

（4）迁移模型：

将在 Isaac Sim 中训练好的机器人模型和策略导出，并导入到 MuJoCo 环境中。确保质量、惯性矩阵、摩擦系数等参数一致。

(5) 验证控制策略：

在 MuJoCo 中重新运行训练好的策略，观察其在更精确的物理环境中的表现。如果发现性能下降，可能需要调整控制算法。

(6) 优化策略：

根据 MuJoCo 的反馈结果，微调控制算法，提升其鲁棒性。

2. 从仿真到实际机器人的运动控制迁移与调试 (sim to real)

Sim-to-Real 的目标是将在仿真环境中训练的策略迁移到真实硬件上。其部署流程如下：

(1) 硬件准备：

确保真实机器人具备与仿真模型一致的传感器和执行器配置。

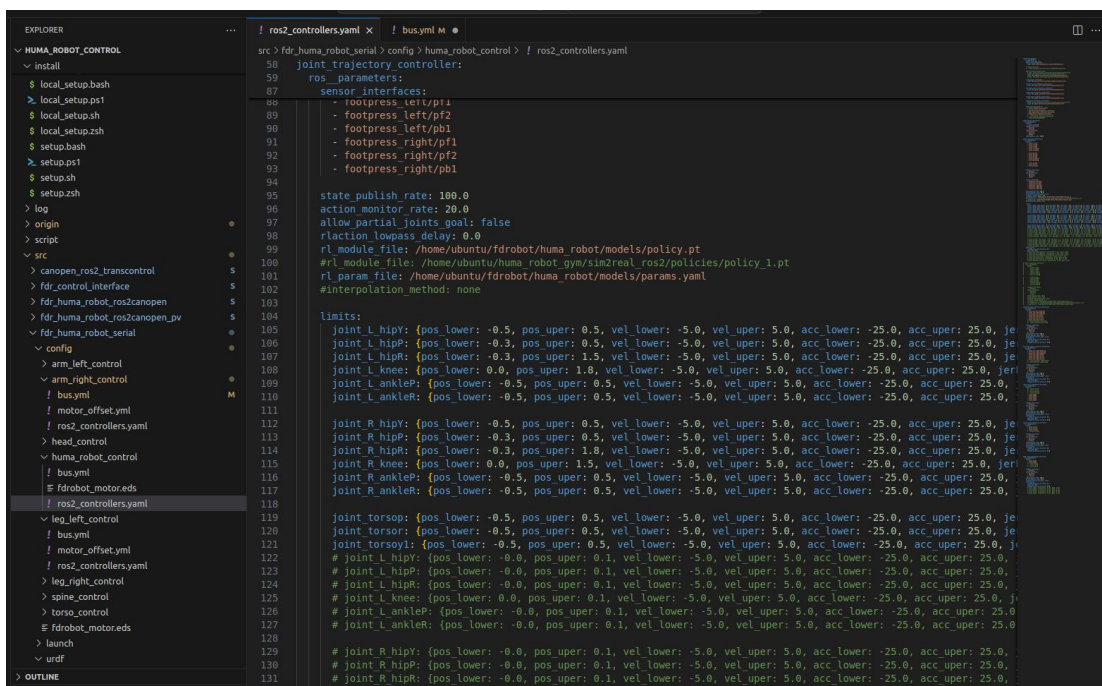
检查电机，imu 等其他软硬件是否有异常。

(2) 迁移策略：

将训练好的控制策略加载到机器人控制器中。

进入

```
src/fdr_huma_robot_serial/config/huma_robot_control  
/ros2_controllers.yaml
```



```
src > fdr_huma_robot_serial > config > huma_robot_control > / ros2_controllers.yaml
58 joint_trajectory_controller:
59   ros_parameters:
60     sensor_interfaces:
61       - footpress_left/pf1
62       - footpress_left/pf2
63       - footpress_left/pb1
64       - footpress_right/pf1
65       - footpress_right/pf2
66       - footpress_right/pb1
67
68   state_publish_rate: 100.0
69   action_monitor_rate: 20.0
70   allow_partial_joints_goal: false
71   rtaction_timeout_delay: 0.0
72   rl_module_file: /home/ubuntu/fdrrobot/huma_robot/models/policy.pt
73   #rl_module_file: /home/ubuntu/huma_robot_gym/sim2real_ros2/policies/policy_1.pt
74   rl_param_file: /home/ubuntu/fdrrobot/huma_robot/models/params.yaml
75   #interpolation_method: none
76
77   limits:
78     joint_L_hipP: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
79     joint_L_hipR: {pos_lower: -0.3, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
80     joint_L_hipR: {pos_lower: -0.3, pos_upper: 1.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
81     joint_L_knee: {pos_lower: 0.0, pos_upper: 1.8, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
82     joint_L_ankleP: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0,
83     joint_L_ankleR: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0,
84
85     joint_R_hipP: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
86     joint_R_hipR: {pos_lower: -0.3, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
87     joint_R_hipR: {pos_lower: -0.3, pos_upper: 1.8, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
88     joint_R_knee: {pos_lower: 0.0, pos_upper: 1.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
89     joint_R_ankleP: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0,
90     joint_R_ankleR: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0,
91
92     joint_torsoP: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
93     joint_torsoR: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, je
94     joint_torsoY: {pos_lower: -0.5, pos_upper: 0.5, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, j
95     # joint_L_hipP: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0,
96     # joint_L_hipR: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0,
97     # joint_L_knee: {pos_lower: 0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, j
98     # joint_L_ankleP: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0
99     # joint_L_ankleR: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0
100
101     # joint_R_hipP: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0,
102     # joint_R_hipR: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0,
103     # joint_R_knee: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0, j
104     # joint_R_ankleP: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0
105     # joint_R_ankleR: {pos_lower: -0.0, pos_upper: 0.1, vel_lower: -5.0, vel_upper: 5.0, acc_lower: -25.0, acc_upper: 25.0
```

在真实环境中进行初步测试，观察其表现。

（3）在线调整：

使用实时数据微调控制策略。

如果发现抖动或者误差，可以通过在线学习或手动干预进行修正。